

examples of how we might use this:

- Say we wanted to model a student in a course. We could have a `Student` class that has instance variables for `name`, `emailAddress`, `currentGrade`, and so on. Each `Student` object we create from this class would have its own set of these instance variables, and the values given to the instance variables would be different for each student.
- Consider a game where we have multiple players. A player could be modeled by a `Player` class with instance variables for `name`, `points`, `health`, `location`, and so on. Each player would have the same capabilities, but the methods could work differently based on the different values in the instance variables.
- Imagine an address book. We could create a `Person` class with instance variables for `name`, `address`, `phoneNumber`, and `birthday`. We could create as many objects from the `Person` class as we want, one for each person we know. The instance variables in each `Person` object would contain different values. We could then write code to search through all the `Person` objects and retrieve information about the one or ones we are looking for.

In future chapters, I will explore this concept of instantiating multiple objects from a single class and give you tools to help manage a collection of objects.

## OOP as a Solution

Toward the end of Chapter 1, I mentioned three problems that are inherent in procedural coding. Hopefully, after working through the examples in this chapter, you can see how object-oriented programming solves all of those problems:

1. A well-written class can be easily reused in many different programs. Classes do not need to access global data. Instead, objects provide code and data at the same level.
2. Object-oriented programming can greatly reduce the number of global variables required, because a class provides a framework in which data